QTransLIFIA: traducción y verificación semántica de circuitos cuánticos desde Quirk hacia ecosistemas de software cuántico

Equipo del Proyecto QTransLIFIA
Laboratorio de Investigación y Formación en Informática Avanzada (LIFIA)

26 de octubre de 2026

Resumen

La computación cuántica se apoya en un ecosistema fragmentado de herramientas de diseño, lenguajes intermedios y *software development kits* (SDK) de ejecución que no comparten una representación común. Esta heterogeneidad dificulta la portabilidad de los circuitos y amenaza la fidelidad semántica del diseño original cuando este debe migrarse entre plataformas. En este trabajo presentamos **QTransLIFIA**, un traductor de circuitos cuánticos que convierte diseños elaborados en la herramienta interactiva *Quirk* hacia *OpenQASM* como lenguaje pivote de representación intermedia y, desde allí, hacia múltiples ecosistemas de destino (IBM Qiskit, AWS Braket y Xanadu PennyLane). A diferencia de una transliteración sintáctica, QTransLIFIA adopta un enfoque de *traducción semántica con preservación de abstracciones*: conserva las compuertas personalizadas definidas por el usuario —tanto las compuestas por circuito como las definidas por matriz—, la disposición columnar y las relaciones de control y objetivo del circuito original. El trabajo se enmarca en la *Ingeniería de Software Cuántica* (*Quantum Software Engineering*, QSE) y articula explícitamente seis atributos de calidad: abstracción, expresividad, verificabilidad, *testing*, escalabilidad y extensibilidad. La fidelidad de la traducción se instrumenta como un artefacto reproducible del propio flujo mediante un verificador semántico integrado basado en cálculo ZX, que compara el grafo derivado de la fuente Quirk con el derivado del OpenQASM generado. Los resultados muestran que un diseño puede traducirse una única vez a OpenQASM y distribuirse simultáneamente a varios *backends*, preservando la intención algorítmica del diseño y habilitando la verificación temprana en la etapa más económica del flujo. QTransLIFIA se distribuye como biblioteca instalable con una interfaz de programación y una interfaz de línea de comandos, y como cuaderno Jupyter para exploración y demostración.

Palabras clave: traducción de circuitos cuánticos; Quirk; OpenQASM; ingeniería de software cuántica (QSE); preservación semántica; compuertas personalizadas; equivalencia por cálculo ZX; verificación y validación; migración multi-ecosistema; Qiskit; AWS Braket; PennyLane.

1. Introducción

La programación de circuitos cuánticos se desarrolla hoy en un ecosistema *multi-lenguaje* y esencialmente *multi-vendor*. El diseño conceptual suele realizarse en herramientas visuales e interactivas como *Quirk* [1]; la simulación, compilación y ejecución se delegan a SDK propietarios o de comunidad como IBM Qiskit [5], AWS Braket [6] o Xanadu PennyLane [7]; y entre ambos extremos circulan lenguajes intermedios de intercambio como OpenQASM [2, 3]. Esta separación de responsabilidades es deseable desde el punto de vista de la ingeniería, pero introduce un riesgo central: la *pérdida de fidelidad semántica* durante la traducción. Un circuito que se traduce incorrectamente —un control desplazado, un objetivo intercambiado, una abstracción aplanada

o una compuerta descartada— produce una transformación unitaria distinta de la diseñada, y el error se propaga silenciosamente a todos los artefactos posteriores.

La *Ingeniería de Software Cuántica* (*Quantum Software Engineering*, QSE) [8] propone tratar los artefactos cuánticos con los mismos principios de calidad que el software clásico: modularidad, trazabilidad, verificabilidad y reproducibilidad. Bajo esta óptica, un traductor de circuitos no debe limitarse a emitir sintaxis válida, sino que debe *preservar* la intención algorítmica del diseño y *demostrar* que lo hace.

QTransLIFIA responde a esta necesidad. Se trata de un traductor que toma circuitos diseñados en Quirk —a partir de su URL con el JSON embebido— y los proyecta sobre OpenQASM, empleado como lenguaje pivote de representación intermedia. Sobre esa base común, el sistema genera código ejecutable para varios ecosistemas de destino y verifica la equivalencia entre el circuito de origen y el traducido mediante cálculo ZX.

Aportes. Este trabajo presenta:

(I) un enfoque de *traducción semántica con preservación de abstracciones*, que conserva las compuertas personalizadas de Quirk como entidades funcionales trasladables, en lugar de aplanarlas;

(II) la instrumentación explícita de seis atributos de calidad de QSE —abstracción, expresividad, verificabilidad, *testing*, escalabilidad y extensibilidad— como decisiones de arquitectura observables;

(III) un *verificador semántico integrado* basado en cálculo ZX [4], que convierte la comprobación de equivalencia en un artefacto reproducible del propio flujo;

(IV) una arquitectura de representación intermedia que permite migrar un único circuito a múltiples *backends* sin reescribir el núcleo de traducción; y

(V) una implementación distribuible como biblioteca (API y CLI) y como cuaderno Jupyter, con una batería de pruebas automatizadas sobre circuitos canónicos.

El resto del documento se organiza y estructura de la siguiente manera: la Sección 2 describe Quirk, OpenQASM y la arquitectura del flujo. La Sección 3 desarrolla los seis atributos de calidad desde la perspectiva QSE. La Sección 4 detalla los mecanismos de verificación y *testing*. La Sección 5 aborda la migración a ecosistemas destino. La Sección 6 discute implicaciones y limitaciones, y la Sección 7 presenta las conclusiones.

2. Antecedentes y arquitectura

2.1. El punto de partida: Quirk

Quirk [1] es un simulador de circuitos cuánticos basado en navegador, orientado a la exploración interactiva. Su unidad de diseño es una matriz de *columnas*; cada columna representa una tajada temporal de operaciones. Un circuito se almacena íntegramente en la URL de la página como un objeto JSON codificado en el fragmento `#circuit=...`. Quirk no se limita a ofrecer compuertas nativas: permite al usuario *definir sus propias compuertas* de dos maneras:

- **por circuito** (`circuit.cols`): una compuerta compuesta por columnas internas que reutilizan otras compuertas (nativas o previamente definidas) que el usuario determina mediante un subrango de segmentos de compuertas seleccionado; y
- **por gates** (`matrix`): una compuerta nombrada, especificada directamente por su matriz unitaria y referenciada en el circuito mediante ese nombre.

Estas definiciones constituyen el *lenguaje expresivo del diseñador*: la forma en que el usuario estructura un algoritmo en bloques reutilizables.

2.2. El lenguaje pivote: OpenQASM

OpenQASM [2, 3] es un lenguaje textual, imperativo y basado en registros cuánticos (`q[...]`), ampliamente soportado por simuladores, transpiladores, compiladores y SDK. Su adopción como *representación intermedia* convierte a QTransLIFIA en un puente entre el dominio de diseño visual (Quirk) y los dominios de ejecución. El sistema soporta dos versiones: OpenQASM 2.0 y OpenQASM 3.0 (esta última por defecto), seleccionables mediante configuración. La versión 2.0 se obtiene por transpilación a un conjunto de primitivas equivalente, mientras que la versión 3.0 preserva además la estructura declarativa de las compuertas definidas por el usuario.

2.3. Arquitectura del flujo

QTransLIFIA se organiza como una tubería de etapas con responsabilidades independientes, de modo que la lógica de traducción resulte auditable y reutilizable. La Tabla 1 resume las etapas principales y su artefacto característico.

Cuadro 1: Etapas del flujo de traducción y artefacto principal.

Etapa	**Artefacto / comprobación**
Análisis del origen	Modelo estructurado del circuito Quirk (dimensiones, compuertas, compuertas personalizadas).
Traducción a OpenQASM	Código OpenQASM (`algorithms_qasm/`) como representación intermedia.
Verificación semántica	Comparación de grafos ZX (Quirk frente a OpenQASM).
Comprobación visual	Capturas y diagramas en PNG, con comparación lado a lado en Markdown.
Migración a destino	Scripts Python ejecutables por ecosistema (`migrated_circuits/`).

El diseño establece una *frontera clara entre dominios*: el modelo visual y columnar de Quirk queda contenido en el analizador y el resolutor de compuertas personalizadas, mientras que la capa de emisión se limita a producir OpenQASM válido. Los detalles de un dominio no contaminan al otro, en aplicación directa del principio de separación de preocupaciones.

La arquitectura evita deliberadamente la duplicación de lógica: la traducción Quirk → OpenQASM, la migración a proveedores, la generación de grafos ZX y las utilidades de carga y dibujo residen en un único módulo de utilidades compartido por las tres formas de ejecución del sistema —biblioteca/API, interfaz de línea de comandos y cuaderno Jupyter—. De este modo, cualquiera de las tres rutas ejecuta exactamente la misma lógica, sin variantes contradictorias.

2.4. Formas de ejecución y contrato de entrada

El flujo funcional es único y se ofrece mediante tres puntos de entrada:

- **Librería (API):** una función de traducción que recibe un *payload* JSON y devuelve el resultado sin tocar disco, y una función de ejecución del flujo completo sobre el directorio de entrada;
- **Línea de comandos (CLI):** invocable como módulo, con parámetros para la entrada, la versión de OpenQASM, los proveedores de destino y los interruptores de cada etapa; y
- **Cuaderno Jupyter:** un flujo exploratorio y visual, útil para demostraciones y para la inspección artefacto por artefacto.

La entrada se especifica como uno o más archivos JSON en los que cada clave es un nombre de algoritmo y cada valor contiene una URL de Quirk, un desplazamiento de índices opcional y una descripción opcional. Los artefactos se agrupan por corrida en un directorio con marca temporal de fecha y hora, con resolución de hora: las ejecuciones dentro de la misma hora se consolidan en el mismo directorio y sobrescriben sus artefactos, mientras que las corridas de horas distintas quedan preservadas como directorios independientes.

3. Atributos de calidad desde la ingeniería de software cuántica

QTransLIFIA se concibe con atributos de calidad explícitos, documentados de manera individual y consolidados como un conjunto coherente. La Tabla 2 los resume y anticipa cómo se materializa cada uno.

Cuadro 2: Atributos de calidad y su materialización en QTransLIFIA.

Atributo	**Materialización**
Abstracción	Traducción semántica con preservación de compuertas personalizadas; separación entre análisis y generación; ocultamiento de la complejidad del formato de Quirk.
Expresividad	Conservación funcional de las compuertas definibles en Quirk (por circuito y por matriz), respetando el punto de partida de diseño.
Verificabilidad	Equivalencia estructural y funcional entre el circuito de Quirk y el OpenQASM generado, apoyada en metadatos y determinismo.
Testing	Ocho baterías desplegadas: análisis estático, suite unitaria, fidelidad numérica de matrices, equivalencia ZX, equivalencia entre backends, corrida extremo a extremo, comprobaciones manuales y aceptación de la distribución publicada.
Escalabilidad	Crecimiento vertical (complejidad del circuito) y horizontal (múltiples ecosistemas) sobre OpenQASM como interfaz estable.
Extensibilidad	Incorporación de nuevas compuertas, versiones de OpenQASM y consumidores sin rediseñar el núcleo.

3.1. Abstracción

La abstracción se manifiesta en tres niveles. Primero, en la *conservación de las abstracciones del usuario*: el traductor no aplana las compuertas personalizadas, sino que las mantiene como entidades identificables en OpenQASM. Segundo, en la *ocultación de la complejidad del formato*: el JSON anidado dentro de la URL de Quirk queda encapsulado por el analizador, que expone al resto del sistema estructuras de datos normalizadas de alto nivel (dimensiones del circuito, inventario de compuertas y de compuertas personalizadas). Tercero, en la *separación entre análisis y generación*: puede analizarse el circuito sin emitir código, lo que habilita inspección, validación y métricas con independencia de la sintaxis generada. Esta separación constituye una frontera explícita entre dos modelos semánticos —el visual y columnar de Quirk y el textual e imperativo de OpenQASM— evitando que los detalles de representación de un dominio se filtren en el otro.

3.2. Expresividad

La expresividad es la capacidad de *conservar el poder expresivo con el que el usuario diseñó el circuito* y trasladarlo, sin pérdida, al lenguaje pivote y al ecosistema destino. El aspecto central es la conservación funcional de las compuertas definibles en Quirk:

- **Compuertas por circuito:** se traducen como declaraciones de compuerta de OpenQASM; el cuerpo interno se deriva de sus columnas y se conserva su estructura en lugar de expandirse

y desaparecer. Las llamadas y las declaraciones coinciden siempre, de modo que la compuerta permanece *trasladable y reutilizable* fuera de Quirk.

- **Compuertas por matriz:** se convierten en la operación OpenQASM equivalente, preservando su comportamiento funcional. Ninguna vía de definición de Quirk queda fuera del alcance del traductor.
- **Composición y anidamiento:** las compuertas pueden componerse jerárquicamente (una compuerta definida puede contener otras), con un límite de profundidad de anidamiento como salvaguarda de robustez. Las instancias multicolumna de una misma definición se desambiguan mediante un sufijo que preserva su contexto de uso, y el orden de las declaraciones se resuelve topológicamente para respetar la regla "declarar antes de usar".

La conservación se comprueba, no se asume: el modelo estructurado del circuito permite auditar que toda compuerta definida fue considerada y mapeada, y el cálculo ZX corrobora la equivalencia semántica de las compuertas trasladadas. El resultado es que los ingenieros cuánticos pueden definir o importar en Quirk sus compuertas personalizadas —ya probadas— sin que la traducción al ecosistema destino constituya una limitante, sino una *transferencia de módulos funcionales*.

3.3. Verificabilidad

La verificabilidad es la capacidad de comprobar que el circuito traducido es equivalente al original, tanto estructural como funcionalmente. El punto de comprobación más sensible se ubica en las *etapas iniciales* del flujo: cuando el circuito existe todavía en su forma de origen. Detectar allí un defecto de traducción es la barrera de control de calidad más temprana y económica, pues cualquier error introducido en esa fase se propaga a todos los artefactos posteriores.

La verificación se estructura en tres niveles complementarios:

Nivel sintáctico. El OpenQASM emitido debe ser conforme al estándar: cabecera, declaraciones de registros válidas, aridad correcta e identificadores válidos. Se resuelve con analizadores OpenQASM existentes.

Nivel estructural. Deben coincidir dimensiones, inventario de compuertas, controles, objetivos, conectividad y jerarquía de compuertas personalizadas entre el circuito analizado y el código emitido. Es totalmente automatizable sin simulación.

Nivel semántico. Ambas representaciones deben producir la misma acción unitaria sobre el espacio de estados, salvo fase global. Se comprueba de forma nativa mediante el cálculo ZX (Sección 4).

La preservación estructural es condición *necesaria pero no suficiente* de la equivalencia funcional: garantiza que no se perdieron piezas, mientras que la comprobación definitiva de la unitaria la aporta el nivel semántico. La verificabilidad se apoya, además, en el *determinismo* del traductor —una misma URL produce siempre el mismo OpenQASM—, que habilita pruebas de regresión robustas, y en la *trazabilidad* de cada instrucción hasta la columna y la definición de compuerta que le dieron origen.

Conviene distinguir con precisión qué garantiza la librería y qué queda del lado del proveedor, porque ello delimita también el alcance de la verificación. El *contrato verificable* es el OpenQASM emitido en el JSON de salida: sobre ese artefacto textual se aplican los niveles sintáctico y estructural, y sobre él se construye la comprobación semántica mediante ZX. La equivalencia Quirk $\leftrightarrow$ OpenQASM es, por tanto, responsabilidad y garantía de QTransLIFIA. En cambio, la carga del OpenQASM y su ejecución dependen del proveedor y ocurren *fuera* del contrato: materializar el circuito y ejecutarlo son operaciones específicas de cada ecosistema, con

analizadores y capacidades heterogéneos, por lo que se realizan de forma conjunta y aislada en un entorno propio de cada proveedor, típicamente mediante los scripts migrados que genera la propia librería. En consecuencia, la equivalencia funcional entre el OpenQASM y el *backend* solo es contrastable dentro del entorno del proveedor, y su resultado depende tanto de la fidelidad de la traducción —ya verificada en origen— como de las particularidades del SDK y del *backend*. Mantener clara esa frontera permite atribuir cualquier discrepancia observada en la ejecución a la traducción o al proveedor, sin ambigüedad sobre dónde reside la responsabilidad.

3.4. Testing

El flujo no es una transformación monolítica, sino una tubería de etapas, y cada transformación es una oportunidad de introducir un defecto. QTransLIFIA articula tres familias de estrategias complementarias:

(1) **Verificación estática:** aserción de invariantes sobre el modelo estructurado —número efectivo de qubits, número de columnas, inventario y conteo de compuertas, compuertas personalizadas, geometría del `offset` y preservación de parámetros— contrastados con el código emitido. Es barata, determinista y totalmente automatizable, por lo que se aplica en cada corrida.

(2) **Equivalencia semántica por cálculo ZX:** comparación automática de la acción unitaria de ambas representaciones mediante grafos ZX, que supera la comprobación estructural al detectar estados de qubits distintos, controles duplicados o expansiones incorrectas de compuertas personalizadas.

(3) **Comprobación visual rápida:** capturas del circuito de Quirk, diagramas del OpenQASM generado y comparaciones lado a lado, aptas para inspección humana inmediata en circuitos sencillos y como *sanity check* en circuitos grandes.

Ninguna estrategia es suficiente por sí sola: la estática garantiza que no se pierden piezas, la semántica aporta la garantía formal de equivalencia y la visual ofrece la lectura inmediata que acelera la detección de defectos.

Sobre esas tres familias se despliega el arsenal de pruebas del proyecto, organizado en *ocho baterías* con un alcance, un procedimiento de ejecución y una garantía explícita cada una. La Tabla 3 las resume. La disciplina de fondo es que cada batería *congela un contrato*: debe poder fallar —es decir, ser capaz de detectar un defecto real— y su documentación se actualiza en la misma entrega en que se modifica el comportamiento congelado.

Los procedimientos exactos son los siguientes:

```
python tools/check_static.py qtrans test
python -m unittest discover -s test -p "test_*.py"
python tools/verify_matrix_gates.py
python tools/verify_backend_equivalence.py
python -m qtrans --input-mode batch --input-file ""
```

Las baterías A, B, C, E, F y H son automatizables, repetibles y constituyen el criterio de aceptación del proyecto; la G acompaña el flujo de trabajo del usuario. Tres precisiones sobre su alcance son pertinentes. La batería C existe porque las compuertas definidas por matriz son el punto donde la traducción puede perder información —el traductor no copia la matriz, emite una *descomposición equivalente*— y una equivalencia de texto no basta: la batería reconstruye la unitaria a partir del cuerpo OpenQASM realmente emitido y la contrasta con la matriz de origen y con un camino de descomposición independiente, de modo que el residuo observado es el redondeo con el que la propia herramienta de origen serializa las matrices y no una pérdida de fidelidad. La batería E verifica una propiedad que las anteriores no cubren: el usuario puede

Cuadro 3: Baterías de testing desplegadas en QTransLIFIA.

Batería	**Ámbito**	**Procedimiento**	**Garantía**
A. Análisis estático	Integridad referencial entre módulos y ausencia de imports muertos	Script de análisis	Ninguna referencia rota ni dependencia colgando
B. Suite unitaria	Contrato funcional del traductor, de la API y del pipeline: 131 pruebas en once archivos	Línea de órdenes	Detección de regresiones sobre el comportamiento congelado
C. Fidelidad numérica	Compuertas personalizadas con campo *matrix*: matriz de origen frente a la descomposición emitida	Script de verificación	Residuo $\leq 10^{-6}$ (medido $\approx 5 \times 10^{-8}$)
D. Equivalencia ZX	Acción unitaria de la traducción y de las sondas de control	Etapa ZX del pipeline	Veredicto por circuito en el reporte de comparación
E. Equivalencia entre backends	Ruta nativa del parser frente a la ruta del script migrado	Script de verificación	Ambas rutas describen el mismo circuito
F. Corrida extremo a extremo	Integración completa y artefactos de la corrida	Línea de órdenes	Artefactos completos y cero errores por etapa
G. Comprobaciones manuales	Exploración y documentación de la interfaz de uso	Scripts y cuadernos	Evidencia cualitativa y ejemplos
H. Aceptación de la distribución	Consumo externo del artefacto publicado en PyPI: traducción, carga y simulación en un entorno aislado	Script del subproyecto	El *wheel* instalado reproduce el contrato sin el código fuente local

materializar el circuito mediante el parser nativo de cada ecosistema o mediante el script migrado, y ambas rutas deben describirlo de igual modo, lo que exige canonicalizar las representaciones equivalentes que cada SDK ofrece para la fase y para las compuertas controladas. Finalmente, la batería D cubre un subconjunto del corpus: las post-selecciones nativas de Quirk no son unitarias, de modo que la equivalencia unitaria no está definida para ellas y quedan fuera de la comparación, con un veredicto explícito —no comprobado— en la API de biblioteca y con el incidente registrado en la etapa ZX del pipeline, en lugar de un falso negativo.

La batería H cierra el ciclo sobre el artefacto que recibe el usuario. El subproyecto aislado `qtrans_demo` instala `qtranslifia==0.2.0` desde PyPI en su propio entorno virtual, comprueba que `qtrans` se importe desde allí y ejecuta dos circuitos con compuertas definidas por matriz a través de `translate_circuit`; después simula los circuitos Qiskit resultantes con 1024 disparos. Las guardas sobre `sys.prefix` y `qtrans.__file__` impiden que la corrida use el código fuente del repositorio. El resultado de referencia es 2/2 circuitos traducidos y simulados, sin errores de etapa. Su alcance es deliberadamente acotado —un proveedor, dos circuitos y un simulador local—, pero cubre el empaquetado, la importación, la resolución de dependencias y la frontera traducción–carga–ejecución en un entorno limpio. La corrida se invoca con el intérprete del entorno del demo y el script del subproyecto, desde la raíz del repositorio.

3.5. Escalabilidad

La escalabilidad se aborda en dos direcciones. La *vertical* corresponde al crecimiento en la complejidad del circuito: más qubits, más columnas y jerarquías de compuertas personalizadas más profundas. Se soporta mediante un cálculo de alturas consciente de la geometría —que determina la altura ocupada por cada compuerta multi-qubit o compuesta y evita asignaciones incorrectas de índices— y mediante un cálculo dinámico del número efectivo de qubits y del desplazamiento, de modo que la asignación a registros permanezca coherente aunque las abstracciones añadan qubits virtuales o modifiquen la altura efectiva.

La escalabilidad *horizontal* corresponde a la integración con múltiples ecosistemas destino. El eje central es OpenQASM como interfaz estable: al emitir este formato intermedio, el traductor queda agnóstico al *backend*. Nuevos consumidores — simuladores, transpiladores, *toolchains* de validación— pueden integrarse como consumidores puros del OpenQASM generado, y es factible añadir etapas posteriores (optimización, mapeo a topología física, reescritura para dispositivos específicos) que operen sobre el IR sin acoplarse a la lógica de análisis de Quirk.

3.6. Extensibilidad

La extensibilidad es la capacidad de evolucionar sin rediseñar la arquitectura central. Se fundamenta en tres principios: separación de responsabilidades —análisis, expansión de compuertas personalizadas y emisión están claramente desacoplados—; abstracción mediante mapeos, donde una tabla de compuertas y un modelo estructurado del circuito facilitan añadir elementos sin alterar los mecanismos existentes; y centralización de la emisión, que permite sustituir o extender la capa de salida de forma aislada.

Estos principios habilitan cuatro líneas de evolución:

- **Evolución de Quirk:** el mapeo por identificador y nombre absorbe cambios de nomenclatura o nuevos identificadores, y la extracción explícita de compuertas personalizadas hace que el soporte a nuevas abstracciones sea implícito.
- **Nuevas compuertas y funcionalidades:** incorporar compuertas nativas localiza el cambio en la capa de mapeo y emisión, sin afectar el cálculo de alturas, el mecanismo de expansión ni el análisis. La información estructurada del circuito habilita, además, funcionalidades futuras —validación, métricas, reportes estructurados— sin acoplarlas a la generación.

- **Nuevas versiones de OpenQASM:** la emisión aislada permite introducir un nuevo generador reutilizando íntegramente el análisis y la expansión de abstracciones. La coexistencia de OpenQASM 2.0 y 3.0 es una manifestación temprana de esta capacidad.
- **Nuevos ecosistemas destino:** al adoptar un IR canónico, los nuevos destinos se integran como adaptadores o consumidores externos, sin proliferar lógica específica de Quirk por *backend*.

El *contrato principal* de la herramienta es la emisión del OpenQASM textual, devuelto en el campo `qasm` del JSON de salida. Ese texto —y no un objeto de circuito dependiente del proveedor— es la interfaz que la librería garantiza, versiona y mantiene estable, por ser serializable, diffeable y portable; un objeto de circuito nativo (`QuantumCircuit` de Qiskit, `Circuit` de Braket, `QNode` de PennyLane) es, en cambio, una estructura en memoria atada a la versión de librería y al entorno local. La librería ofrece además, en `output.migrated_code`, un script Python por proveedor que sirve de puente: el usuario lo ejecuta en su propio entorno para materializar el circuito del ecosistema elegido.

La carga efectiva del OpenQASM y, sobre todo, la ejecución no forman parte de ese contrato, porque *dependen intrínsecamente del proveedor* y son heterogéneas entre ellos: el analizador nativo de Qiskit cubre la totalidad de los casos, Braket no conoce la compuerta `p` y requiere respaldo, y PennyLane no dispone de un analizador OpenQASM nativo útil. Por ello, la carga (*loads*) y la ejecución —asignación de *shots*, *backend*, transpilación a topología de dispositivo— se realizan habitualmente de forma *conjunta y aislada en un entorno propio de cada proveedor*, fuera del núcleo del traductor. Esta separación es, precisamente, lo que hace extensible al sistema: incorporar un nuevo destino no obliga a modificar el núcleo ni a acoplarlo a ese SDK, y la traducción permanece estable mientras el ecosistema destino evoluciona de forma independiente.

4. Verificación y validación

4.1. El verificador semántico integrado

El componente más distintivo de QTransLIFIA es su verificador semántico basado en cálculo ZX, que se apoya en la librería PyZX [4]. El proceso consta de tres pasos: se construye un grafo ZX *directamente desde la fuente Quirk*; se construye un segundo grafo *desde el OpenQASM generado*; y se comparan ambos grafos por circuito. La estrategia de comparación se adapta al tamaño del circuito: para circuitos pequeños se reduce cada grafo y se comparan sus tensores; para circuitos grandes se verifica la igualdad de los circuitos asociados considerando la fase global. El resultado por circuito —*equivalente*, *no equivalente* o *no comprobado*— se registra como un reporte auditable.

La regla de equivalencia y el criterio de decisión residen en un único punto del sistema, compartido por las tres formas de ejecución, de modo que el veredicto es idéntico independientemente de la ruta empleada. Además, el verificador se protege con un límite de tiempo por comparación: cuando la reducción de un grafo resulta computacionalmente intratable, el circuito se marca como *no comprobado* en lugar de bloquear el flujo. Este mecanismo convierte la comprobación semántica —el nivel más fuerte de verificación— en un *artefacto reproducible del propio pipeline*, en línea con la exigencia de la QSE de instrumentar la verificación como parte del proceso de desarrollo y no como una actividad manual posterior.

4.2. Estrategias automatizables

La separación entre análisis y generación habilita estrategias de verificación automatizada adicionales: el contraste entre el inventario de compuertas del circuito analizado y las instrucciones efectivamente emitidas; la comprobación de invariantes (*property-based testing*) sobre dimensiones, número de controles y rango de índices; las pruebas metamórficas, que exigen invariancia

ante transformaciones que no alteran el circuito; y la validación sintáctica con consumidores externos. El determinismo del traductor sustituye, en estas estrategias, la necesidad de comparar contra una referencia móvil: la comparación contra una salida fija es estable a lo largo del tiempo.

4.3. Estado de la validación

La batería de pruebas del repositorio ejercita los modos de entrada individual y por lotes, circuitos canónicos (estados de Bell, compuertas de Toffoli, compuertas personalizadas simples y anidadas), el desplazamiento de índices y la opción de preservación o expansión de compuertas. Concretamente, la suite reúne **131 pruebas en once archivos**: pipeline y línea de órdenes, contrato de la API de biblioteca (tanto en su forma serializable como en la forma que devuelve objetos nativos de cada ecosistema), convención de orden de qubits verificada en los tres destinos, fidelidad de las compuertas definidas por matriz —incluida una barrida exhaustiva del corpus que compara la unitaria emitida con la matriz de origen, fase global incluida—, sondas de control de eje X/Y y post-selecciones, canonicidad de la transformada de Fourier y normalización de URLs de Quirk.

La verificación semántica por cálculo ZX se ejecuta sobre las colecciones de algoritmos de prueba, que abarcan algoritmos principales, variantes populares y diseños específicos de la transformada cuántica de Fourier y su inversa. El flujo completo —incluidas la generación de imágenes, la comparación visual, los grafos ZX y la migración a proveedores— se ejecuta de extremo a extremo tanto desde la línea de órdenes como desde el cuaderno, sin intervención manual: sobre el corpus principal produce 39 archivos OpenQASM, 39 capturas del circuito original, 39 diagramas, 39 comparaciones lado a lado y 39 scripts migrados, sin errores por etapa. La suite se ejecuta completa sin fallos (131/131); la aserción de idempotencia de la canonicalización de URLs valida ahora el circuito parseado y no el escapado Unicode del texto, que era el origen de la discrepancia.

Sobre el corpus completo —134 algoritmos en seis registros, con las tres etapas de verificación simultáneas— la corrida produce 134 archivos OpenQASM, 134 capturas del circuito original, 134 diagramas, 134 comparaciones lado a lado y 402 scripts migrados, con las 134 imágenes de la herramienta de composición de IBM. La verificación semántica por cálculo ZX compara 77 de esos circuitos y obtiene *equivalencia* en los 77; los 57 restantes no admiten comparación —post-selecciones no unitarias, compuertas escalares globales y una matriz de cuatro por cuatro— y quedan registrados como incidencias de esa etapa, sin afectar a la traducción. El informe de equivalencia no omite esos casos: lista una fila por circuito analizado, con el motivo de cada no comparabilidad y una leyenda que explica el significado de cada veredicto, de modo que la ausencia de una prueba de equivalencia sea siempre visible y explicada. La propia corrida E2E detectó un defecto real que la suite unitaria no podía ver: en la consola de Windows, los símbolos del mensaje de error de la etapa ZX no son codificables y abortaban el informe de equivalencia; la impresión se hace ahora a prueba de codificación y la corrección queda congelada con pruebas de regresión.

Como cierre del ciclo, la batería H ejecuta el artefacto publicado, no el árbol de fuentes: el subproyecto aislado instala `qtranslifia==0.2.0` desde PyPI en su propio entorno, verifica que `qtrans` proceda de ese entorno y traduce y simula dos circuitos con compuertas definidas por matriz (1024 disparos cada uno), sin errores de etapa. Es una comprobación de caja negra del empaquetado y de la frontera traducción–carga–ejecución, no un sustituto de las baterías anteriores.

Los límites de la validación son explícitos: la equivalencia se establece salvo fase global; la verificación ZX puede resultar intratable en circuitos con muchas compuertas multi-controladas, de lo que resulta un veredicto de *no comprobado* con límite de tiempo; las post-selecciones, por no ser unitarias, quedan fuera de la comparación semántica; y la confirmación final frente a un dispositivo real corresponde al proveedor, no al traductor.

5. Migración a ecosistemas destino

Sobre la representación intermedia OpenQASM, QTransLIFIA genera scripts Python ejecutables, uno por algoritmo y ecosistema. La Tabla 4 resume los destinos soportados y sus particularidades.

Cuadro 4: Ecosistemas de destino soportados.

Ecosistema	**Particularidad de la migración**
IBM Qiskit	Emplea compuertas nativas siempre que la descomposición del circuito corresponda a una puerta estándar; produce además diagramas con el estilo de IBM Quantum Composer.
AWS Braket	Requiere el SDK de Amazon Braket; el cargador nativo se complementa con un respaldo por migración para cubrir todo el OpenQASM que genera el proyecto.
Xanadu PennyLane	Se materializa siempre a través del script migrado, generando circuitos ejecutables con un número configurable de disparos.

La migración se beneficia de la preservación de abstracciones: las compuertas definidas por el usuario llegan al SDK destino con su semántica intacta —recreadas como compuertas personalizadas— o reducidas a primitivas equivalentes por el propio SDK. Un mismo circuito puede migrarse una única vez a OpenQASM y distribuirse a varios ecosistemas simultáneamente, de modo que la expresividad conservada rinde en todos los destinos a la vez. La carga del circuito OpenQASM por proveedor también reside en un único punto compartido, de forma que las tres formas de ejecución interpretan el IR de manera idéntica.

6. Discusión

El aporte conceptual de QTransLIFIA no está en la conversión de sintaxis —resoluble con mapeos locales— sino en sostener dos premisas de ingeniería. La primera es que *la abstracción del usuario es parte de la semántica*: aplanar una compuerta personalizada durante la traducción destruye información de diseño y dificulta el mantenimiento del software cuántico. La segunda es que *la fidelidad debe ser demostrable*: incorporar el verificador semántico al flujo convierte una propiedad deseable en una evidencia reproducible.

Entre las limitaciones que delinean el trabajo futuro se destacan: la verificación ZX puede resultar intratable en circuitos con muchas compuertas multi-controladas, motivo por el cual se emplea un límite de tiempo y un veredicto explícito de *no comprobado*; la equivalencia se establece salvo fase global, lo que es adecuado para la mayoría de los propósitos pero conviene documentar por contexto; y la migración a nuevos ecosistemas, si bien no exige modificar el núcleo, sí requiere implementar un generador específico por destino. Finalmente, la captura visual del circuito original depende de un navegador automatizado, por lo que es la etapa más sensible al entorno de ejecución.

Como líneas de evolución, la arquitectura admite incorporar nuevos ecosistemas como consumidores puros del IR, añadir nuevos generadores para versiones futuras de OpenQASM sin rediseñar el análisis, y enriquecer el modelo estructurado con nuevas métricas de calidad.

7. Conclusiones

QTransLIFIA demuestra que es posible migrar circuitos cuánticos entre plataformas preservando la *intención algorítmica* del diseño y *demostrando* dicha preservación como parte del propio flujo. Al combinar traducción semántica con preservación de abstracciones, OpenQASM como representación intermedia estable y un verificador semántico integrado basado en cálculo

ZX, el sistema materializa los seis atributos de calidad que la Ingeniería de Software Cuántica promueve: *expresividad* y *abstracción* definen qué se conserva; *verificabilidad* y *testing* garantizan cómo se comprueba; y *escalabilidad* y *extensibilidad* delimitan hasta dónde puede crecer.

Tres resultados sostienen esta conclusión. Primero, las compuertas personalizadas definibles en Quirk —por circuito o por matriz— sobreviven a la traducción como entidades funcionales, reutilizables y auditables, en lugar de aplanarse. Segundo, la equivalencia entre el circuito diseñado y el traducido se convierte en un artefacto reproducible del pipeline, no en una comprobación manual posterior. Tercero, un único circuito puede migrarse a varios ecosistemas sin reescribir el núcleo, gracias a una arquitectura modular desacoplada y a un lenguaje pivote ampliamente soportado.

Estos tres resultados convergen en la premisa central del proyecto: **el circuito que el usuario diseñó en Quirk es el circuito que se traduce, se verifica y se ejecuta** en cualquiera de los ecosistemas de destino. Al sentar esta premisa sobre principios explícitos de ingeniería de software cuántica, QTransLIFIA se posiciona como una solución preparada para evolucionar con el ecosistema cuántico, preservando al mismo tiempo estabilidad, mantenibilidad y fidelidad semántica.

Referencias

[1] C. Gidney, *Quirk: a drag-and-drop quantum circuit simulator*. https://algassert.com/quirk. Accedido en 2026.

[2] A. W. Cross, L. S. Bishop, J. A. Smolin y J. M. Gambetta, *Open Quantum Assembly Language*, 2017. https://openqasm.com.

[3] A. W. Cross *et al.*, *OpenQASM 3: A broader and deeper quantum assembly language*, ACM Transactions on Quantum Computing, 2022.

[4] A. Kissinger y J. van de Wetering, *PyZX: A Python library for quantum circuit rewriting and simplification*, 2020. https://pyzx.readthedocs.io.

[5] Qiskit Development Team, *Qiskit: an open-source framework for quantum computing*. https://www.ibm.com/quantum/qiskit. Accedido en 2026.

[6] Amazon Web Services, *Amazon Braket: servicio de computación cuántica gestionada*. https://aws.amazon.com/braket/. Accedido en 2026.

[7] Xanadu, *PennyLane: cross-platform Python library for quantum machine learning and quantum computing*. https://pennylane.ai. Accedido en 2026.

[8] M. Piattini, G. Peterssen y R. Pérez-Castillo, *Quantum Software Engineering*, Springer, 2022.